Academic Evidence Investigator

Harnessing the Power of Agentic AI to Discover, Evaluate, and Synthesise Academic Evidence.

Design Proposal for an LLM-Powered Multi-Agent **Academic Evidence Investigator**.

A report produced by **Ruwan Perera**, **Tugba Unal**, **Isaac Essam Shalaan** and **Luis Enrique Torres Zambrano**.

Word Count: Exactly 1,000 (Excluding Cover Page, Table of Contents and Reference List)

Table of Contents

1. **Purpose and Scope** 2
2. **System Requirements and Assumptions** 2
3. **System Architecture and Design Decisions** 3
4. **Agent-Oriented Design Methodology** 6
5. **Development and Evaluation Approach** 7
6. **Challenges, Limitations and Mitigations** 7
7. **Conclusion** 8
8. **References** 8

1. Purpose and Scope

This project is designed for a university Graduate School or Postgraduate Research Support Unit supporting postgraduate students and early-stage researchers. Users must refine research questions, locate relevant literature, assess evidence quality and relevance, and synthesise findings. These activities are time-consuming when large volumes of literature are involved. LLM-based agents are increasingly being explored for planning, decision-making and autonomous task execution (Wang *et al.*, 2024).

The proposed Academic Evidence Investigator will accept a high-level research question, such as "*Does generative AI improve university students' academic performance?*", decompose it into tasks, retrieve scholarly information, evaluate the evidence and produce a structured research briefing. It is intended to support rather than replace the researcher and will only identify possible under-researched areas within the material retrieved.

2. System Requirements and Assumptions

The proposed implementation requires:

- **Python 3.11** or above for core application logic and deterministic processing.
- **LangGraph** for agent orchestration, shared state and conditional workflow.
- **Qwen2.5-7B-Instruct** LLM accessed through **Hugging Face** with the model kept configurable.

- **Semantic Scholar** and **Crossref** APIs for academic search and bibliographic metadata.
- **Requests/HTTPX**, **Pandas** and **Pydantic** for API access, cleaning and structured validation.
- **JSON** as the interchange format between the agent workflow and application layer.
- **SQLite** for prototype persistence, with **PostgreSQL** as a future option.
- **Flask**, **HTML** and **CSS** for the user-facing interface.
- **PyTest**, **Git/GitHub** and **VSCode** for testing, collaboration and development.

Internet access is assumed for retrieval. Academic API coverage may be incomplete, so the system will remain a research aid rather than an authoritative or exhaustive source.

3. System Architecture and Design Decisions

The system will use a supervisor-based multi-agent architecture with four specialist roles:

- **Planner/Supervisor Agent**
  Interprets the research question, creates subtasks and generates search queries.
- **Retrieval Agent**
  Queries Semantic Scholar and Crossref, while deterministic Python functions handle duplicate removal, metadata cleaning and basic ranking.

- **Evidence Analysis/Critic Agent**

  Interprets findings, methods, limitations and conflicting evidence. If evidence is weak or unbalanced, control returns to the Planner for further searching.

- **Reporting Agent**

  Once the available evidence is sufficiently relevant and balanced, the Report Agent produces the final structured briefing and saves the output to the database.

Each agent has a distinct responsibility while contributing to shared research state. This supports coordinated task decomposition and reflects recent LLM-based multi-agent research (Guo *et al.*, 2024) and the goal-directed view of autonomous agents described by Wooldridge (2009).

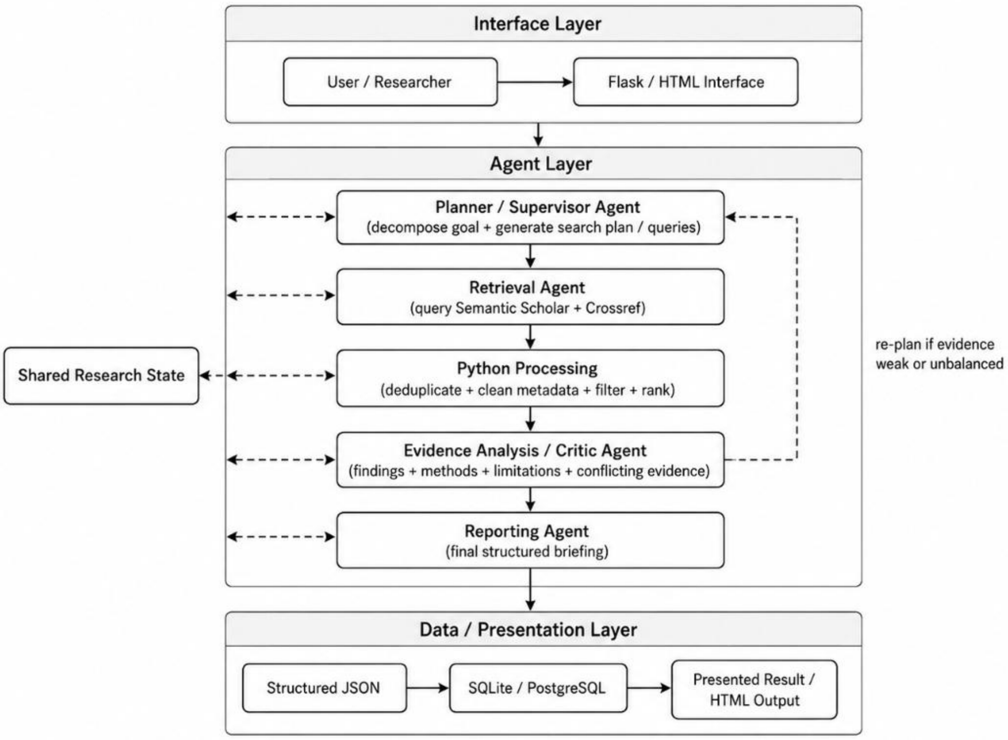
Interface Layer
User / Researcher
Flask / HTML Interface
Agent Layer
Planner / Supervisor Agent
(decompose goal + generate search plan / queries)
Retrieval Agent
(query Semantic Scholar + Crossref)
Shared Research State
Python Processing
(deduplicate + clean metadata + filter + rank)
re-plan if evidence weak or unbalanced
Evidence Analysis / Critic Agent
(findings + methods + limitations + conflicting evidence)
Reporting Agent
(final structured briefing)
Data / Presentation Layer
Structured JSON
SQLite / PostgreSQL
Presented Result / HTML Output

Figure 1: Proposed layered multi-agent system architecture.

The design combines reasoning with external actions, consistent with ReAct's integration of LLM reasoning and external information sources (Yao *et al.*, 2023). The Critic-to-Planner loop is informed by Reflexion, where feedback can improve subsequent decisions (Shinn *et al.*, 2023).

LangGraph is preferred because the workflow requires shared state, conditional routing and controlled re-planning. Its documented orchestrator-worker and evaluator-optimiser patterns closely match the proposed workflow (LangChain, no date). CrewAI was considered, but LangGraph provides more explicit control over state transitions and feedback loops.

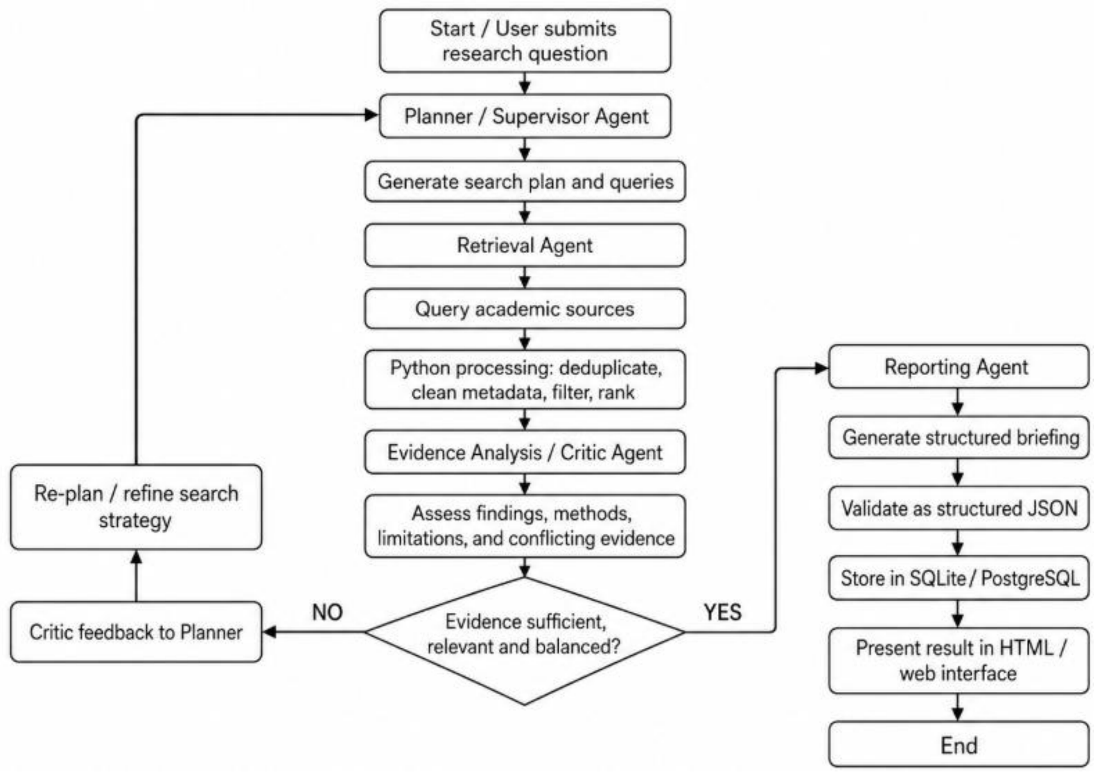
Start / User submits research question
Planner / Supervisor Agent
Generate search plan and queries
Retrieval Agent
Query academic sources
Python processing: deduplicate, clean metadata, filter, rank
Evidence Analysis / Critic Agent
Assess findings, methods, limitations, and conflicting evidence
Evidence sufficient, relevant and balanced?
NO
Critic feedback to Planner
Re-plan / refine search strategy
YES
Reporting Agent
Generate structured briefing
Validate as structured JSON
Store in SQLite / PostgreSQL
Present result in HTML / web interface
End

Figure 2: Activity flow showing critique, re-planning and completion logic.

Retrieval is deliberately separated from generation. Lewis *et al.* (2020) show that retrieval-augmented generation can improve knowledge-intensive tasks by grounding generation in external information. Retrieved metadata and source links will therefore remain associated with generated findings. Final results will be validated as structured JSON, stored in SQLite and presented through a separate web layer. This allows output formats to change without redesigning the agent workflow.

4. Agent-Oriented Design Methodology

The design is informed by the Belief-Desire-Intention (BDI) model, in which beliefs represent available knowledge, desires represent objectives and intentions represent committed plans (Archibald *et al.*, 2024). In the proposed system, shared research state represents available knowledge, each specialist agent has a defined objective, and the Planner maintains and revises the intended research strategy as new evidence becomes available. These BDI concepts support clear goals and adaptive behaviour, while agent communication is handled through LangGraph's shared state and structured transitions rather than formal Agent Communication Languages such as FIPA-ACL or KQML. This reduces communication overhead and implementation complexity while keeping coordination explicit and easier to trace within a centrally orchestrated workflow.

5. Development and Evaluation Approach

Development will follow an iterative Agile approach, supporting incremental implementation, feedback and adaptation (Lutwama *et al.*, 2024). The team will first build a basic end-to-end workflow for one research question, then add criticism, re-planning and database-backed presentation.

Each component will be tested independently before integration. A fixed JSON schema will make intermediate outputs inspectable, while an audit trail will record searches performed, papers retained and reasons for re-planning. Testing will include duplicated records, missing metadata, weak searches and conflicting evidence. A maximum number of search cycles will prevent uncontrolled loops.

6. Challenges, Limitations and Mitigations

LLM hallucination is a risk because outputs may appear plausible without sufficient support. Self-reflection can reduce hallucination but does not eliminate it (Ji *et al.*, 2023), while retrieval-augmented systems may fail when evidence is incomplete, irrelevant or misleading (Huang *et al.*, 2025). Findings will remain linked to retrieved sources and checked by the Critic Agent.

Biased or incomplete retrieval will be mitigated through multiple sources and alternative search terms. The Critic will identify contradictory or inconclusive findings. Duplicate or inconsistent metadata will be resolved using DOI, title and author matching. Context and cost limits will be managed by filtering papers before LLM analysis and prioritising abstracts. The system will identify research opportunities only within retrieved evidence, rather than claim definitive research gaps.

Other challenges include token and API costs, latency, security and privacy. Repeated LLM and API calls may increase costs and delays. User queries, research data and API credentials require protection through secure storage and careful data handling.

7. Conclusion

The proposed design preserves the genuinely agent-based elements of the problem, including planning, delegation, evaluation and re-planning, while assigning predictable operations to conventional software. Separating reasoning, retrieval, persistence and presentation creates a modular architecture that is practical to test and extend. It also retains traceability between generated findings and scholarly evidence, enabling the system to provide a useful research briefing while acknowledging the limitations of automated academic evidence gathering.

8. References

Archibald, B., Calder, M., Sevegnani, M. and Xu, M. (2024) ‘Quantitative modelling and analysis of BDI agents’, *Software and Systems Modeling*, 23, pp. 343–367.

Available at: https://doi.org/10.1007/s10270-023-01121-5 (Accessed: 28 August 2026).

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., Wiest, O. and Zhang, X. (2024) ‘Large Language Model Based Multi-agents: A Survey of Progress and Challenges’, *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, Survey Track, pp. 8048–8057. Available at: https://doi.org/10.24963/ijcai.2024/890 (Accessed: 28 August 2026).

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B. and Liu, T. (2025) ‘A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions’, *ACM Transactions on Information Systems*, 43(2), Article 42, pp. 1–55. Available at: https://doi.org/10.1145/3703155 (Accessed: 28 August 2026).

Ji, Z., Yu, T., Xu, Y., Lee, N., Ishii, E. and Fung, P. (2023) ‘Towards Mitigating LLM Hallucination via Self Reflection’, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 1827–1843. Available at: https://doi.org/10.18653/v1/2023.findings-emnlp.123 (Accessed: 28 August 2026).

LangChain (no date) ‘Workflows and agents’, *LangChain Documentation*. Available at: https://docs.langchain.com/oss/python/langgraph/workflows-agents (Accessed: 28 August 2026).

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. and Kiela, D. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', *Advances in Neural Information Processing Systems*, 33, pp. 9459–9474. Available at: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html (Accessed: 28 August 2026).

Lutwama, P., Dlulane, M., Pillay, T., Hassan, F.S. and Grobbelaar, S. (2024) 'Agile: advantages, disadvantages, enablers, and barriers', *South African Journal of Industrial Engineering*, 35(4), pp. 66–76. Available at: https://doi.org/10.7166/35-4-3058 (Accessed: 28 August 2026).

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. and Yao, S. (2023) 'Reflexion: Language Agents with Verbal Reinforcement Learning', *Advances in Neural Information Processing Systems*, 36, pp. 8634–8652. Available at: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html (Accessed: 28 August 2026).

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W.X., Wei, Z. and Wen, J. (2024) 'A survey on large language model based autonomous agents', *Frontiers of Computer Science*, 18, article

186345. Available at: https://doi.org/10.1007/s11704-024-40231-1 (Accessed: 28 August 2026).

Wooldridge, M. (2009) *An Introduction to MultiAgent Systems*. 2nd edn. Chichester: John Wiley & Sons. Available at: https://www.wiley-vch.de/en/areas-interest/computing-computer-sciences/an-introduction-to-multiagent-systems-978-0-470-51946-2 (Accessed: 28 August 2026).

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R. and Cao, Y. (2023) 'ReAct: Synergizing Reasoning and Acting in Language Models', *The Eleventh International Conference on Learning Representations (ICLR 2023)*. Available at: https://openreview.net/forum?id=WE_vluYUL-X (Accessed: 28 August 2026).